Davante

18

ENTORNOS DE DESARROLLO

Control de versiones y documentación

ÍNDICE

/ 1. Introducción y contextualización práctica 3

/ 2. Concepto del sistema de control de versiones 4

/ 3. Desarrollos colectivos 4

/ 4. Caso práctico 1: “La herramienta adecuada” 5

/ 5. Documentación 5

5.1. Generación automática de documentación 6

/ 6. Uso de comentarios 6

/ 7. Caso práctico 2: “Documentar siempre” 7

/ 8. Resumen y resolución del caso práctico de la unidad 7

/ 9. Bibliografía 8

© MEDAC 978-84-18983-00-9
Reservados todos los derechos. Queda rigurosamente prohibida, sin la autorización escrita de los titulares del copyright, bajo las sanciones establecidas en las leyes, la reproducción, transmisión y distribución total o parcial de esta obra por cualquier medio o procedimiento, incluidos la reprografía y el tratamiento informático.

OBJETIVOS

Conocer el concepto de sistema de control de versiones.

Conocer lo que es un repositorio.

Comprender la necesidad de crear documentación.

Conocer las herramientas de documentación más utilizadas.

/ 1. Introducción y contextualización práctica

En un mundo en el que el desarrollo software cada vez se realiza de una forma más ubicua, es importante disponer de las herramientas necesarias para poder sincronizar diferentes equipos en diferentes partes del mundo. Al tener cada vez sistemas más complejos, es necesario disponer de elementos que permitan controlar el código y sincronizar equipos de manera sencilla. Además, es necesario que múltiples equipos puedan trabajar en diferentes partes del código a la vez. Como los sistemas crecen rápidamente, también es necesario poder ser ágiles a la hora de crear tanto la documentación a entregar a los clientes, como la documentación que va a servir al equipo de desarrollo para mantener el código en un futuro. Por ello, en este tema hablaremos de los sistemas de control de versiones y de la generación de documentación del código fuente.

Con estas dos herramientas seremos capaces de trabajar con equipos interdisciplinares y tener un código fácilmente documentado y trazable a lo largo del tiempo. Veremos que existen diferentes aspectos a considerar cuando trabajamos con sistemas de control de versiones y que existen también distintos tipos de herramientas para la generación de documentación.



Fig. 1. La documentación y el control de versiones son fundamentales.

Empezaremos con el tipo de documentación más sencilla que podemos incluir en un código fuente, **los comentarios**.

Escucha el siguiente audio en el que planteamos el caso práctico que iremos resolviendo a lo largo de esta unidad.

Audio Intro. "Sistema de control de versiones"
https://bit.ly/3pUs882

/ 2. Concepto del sistema de control de versiones

En el desarrollo actual del *software* es necesario instaurar **sistemas que permitan controlar y revisar periódicamente el código que se genera.** Dado que los equipos de desarrollo actualmente son relativamente grandes, y el código fuente suele aumentar fácilmente, es de interés instaurar un sistema que permita establecer el control del código y comprobar de forma ágil lo que ha ido sucediendo en el tiempo durante la implementación de nuevas funcionalidades. Este tipo de sistemas reciben el nombre de **Sistema de Control de Versiones (VCS)**. A través de estos sistemas es posible **identificar las partes del código que han ido cambiando tras la implementación de diferentes funcionalidades.**

Como norma general, estos sistemas **permiten establecer diferentes roles a los usuarios para que puedan trabajar en paralelo,** pues no tendría sentido utilizar un sistema de control de versiones que no nos permita crear nuevos usuarios y que todos puedan trabajar con el mismo código fuente.

Cuando hablamos de un sistema de control de versiones tenemos que hablar del concepto de **repositorio de código.** Estos sistemas permiten **definir diferentes tipos de roles para los usuarios**, que influirán en los permisos que tendrán sobre los repositorios. Antes de empezar un proyecto de desarrollo, una buena práctica es definir la estructura que va a tener el repositorio de código para poder albergar el proyecto en cuestión. Cuando se realizan cambios en el código, cada usuario podrá subir sus cambios al repositorio y se marcará como una revisión del código. En el caso de que los diferentes usuarios hayan hecho cambios en la misma parte del código pueden existir conflictos que tendrán que resolverse con la intervención humana.

Una de las **ventajas** que tiene trabajar con sistema de gestión de código es que permiten comprobar fácilmente los cambios que existen y, en caso de conflictos, marcar aquellas partes conflictivas. Los sistemas de control de versiones se pueden integrar en diferentes entornos de desarrollo para facilitar el trabajo a los desarrolladores.



Audio 1. "Alternativas a los sistemas de control de versiones"
https://bit.ly/38nXfDa

/ 3. Desarrollos colectivos

Como hemos comentado anteriormente, los sistemas de control de versiones permiten que **múltiples usuarios puedan trabajar a la vez sobre el mismo código fuente pudiendo resolver fácilmente los conflictos que puedan existir**. Utilizando herramientas de control de versiones es posible trabajar con equipos grandes sobre el mismo código haciendo uniones de código casi de manera automática. Es normal que cuando trabajemos con un sistema de control de versiones y bajo el mismo código, los desarrolladores puedan modificar el mismo archivo durante el proceso de desarrollo.

Los sistemas de control de versiones son suficientemente inteligentes como para saber unir estos cambios siempre y cuando las líneas que han sido modificadas no sean las mismas. En el caso de tener que resolver este tipo de situaciones, se lanzará un conflicto y se solicitará la ayuda del programador en cuestión.

Normalmente, el **flujo de trabajo con un VCS es el siguiente:**

- **Descarga del código.** Como es lógico, lo primero es que el desarrollador pueda contar con el código para poder trabajar con él. Por ello es necesario realizar una primera descarga del código. En este estado no pueden surgir conflictos porque el programador no tiene código local.

- **Realización de modificaciones.** Una vez que el código está disponible, el programador podrá realizar las modificaciones que estime oportunas para implementar la funcionalidad deseada o solucionar los errores encontrados.

- **Subir el código**. Una vez que el código ha sido modificado será necesario actualizar el repositorio con las modificaciones realizadas. Normalmente, antes de modificar el repositorio deberemos actualizar nuestra copia local para ver si tenemos algún conflicto.

Llegados a este punto, ya podemos comentar la existencia de un sistema de control de versiones como ***Git*** (que lo estudiaremos en la próxima unidad). Éste, es uno de los **sistemas de control** de versiones más utilizados en la actualidad. Se trata de un sistema distribuido en el que **cada uno de los usuarios que trabaja en el proyecto posee una copia local de todo el repositorio.** Aquí te dejamos un avance:

Vídeo 1. "Eclipse y Git como VCS"
https://bit.ly/2lDBCZK

/ 4. Caso práctico 1: "La herramienta adecuada"

Planteamiento: Tras la expansión de nuestra empresa, comenzamos a realizar proyectos con desarrolladores que están ubicados en diferentes partes del mundo. El tipo de proyecto que estamos tratando tiene muchas funcionalidades y por lo tanto está organizado en muchos archivos y muchas líneas de código. En este contexto, nuestro jefe nos pide que trabajemos con el código subido en Dropbox y resolvamos las dependencias y conflictos a través de este sistema de Cloud.

Nudo: ¿Crees que es un enfoque correcto?

Desenlace: Según hemos visto en el tema, cuando el número de líneas de código aumenta, puede ser muy complicado realizar su gestión si el equipo de desarrollo es muy grande.

De esta forma, necesitamos utilizar herramientas especializadas para permitir que muchos individuos realicen cambios sobre el mismo código.

Utilizar un sistema como Dropbox puede ser contraproducente ya que el número de veces que un archivo se puede cambiar es muy alto y pueden existir problemas de sincronización y conflictos entre los archivos. Además, Dropbox no ha sido diseñado para este tipo de tareas. Lo correcto sería utilizar un sistema de control de versiones como Git, que permita fácilmente detectar los cambios realizados y los posibles conflictos que podrían surgir. En el caso de Dropbox, no sería posible evitar los conflictos porque varios programadores hayan cambiado la misma parte del código.



Fig. 2. Es necesario resolver correctamente los conflictos del código.

/ 5. Documentación

La documentación de un proyecto es extremadamente importante.

Además, hay que considerar:

- Diferentes etapas del proyecto.
- Diferentes consumidores de la documentación que generemos.

De esta forma, podemos ver que existen al menos tres etapas en la **generación de documentación: previa al desarrollo, durante el desarrollo y después del desarrollo.**

Si tenemos en cuenta los consumidores de esta documentación podríamos distinguir al menos dos roles:

- **Consumidores internos.** Serían aquellos que deben tener mayor cantidad de datos para poder implementar las funcionalidades requeridas.
- **Consumidores externos.** Aquellos que no pertenecen al proyecto y que deben tener un tipo de documentación adaptada.

Tradicionalmente, la documentación para consumidores externos se puede resumir en **manuales de usuario.** En el caso del desarrollo de APIs, la documentación externa se podría ver como un resumen de la funcionalidad ofrecida, indicando los métodos formas de uso.

5.1. Generación automática de documentación

Normalmente, es necesario poder generar documentación del código fuente existente a través de diferentes herramientas. Algunas de estas herramientas **pueden estar integradas en los entornos de desarrollo, o** bien, pueden utilizarse como herramientas **externas que pueden analizar el código y generar la documentación que hayamos incluido.**

Para cada una de las herramientas que queramos utilizar es necesario **conocer la sintaxis** para poder incluir la información de documentación. La forma más extendida de poder introducir esta documentación es mediante la **utilización de etiquetas** que, permiten introducir un tipo de campo determinado que será analizado por la herramienta de generación de documentación. Nótese que, esta documentación simplemente permite documentar el código, y no están diseñadas para generar manuales de usuario. Por ejemplo, se suele utilizar para la documentación de APIs.



Audio 2. "Herramientas para generación de documentación"
https://bit.ly/3hWajTp





Vídeo 2. "Generación de documentación con JavaDoc"
https://bit.ly/2lAl58G



/ 6. Uso de comentarios

En la actualidad, el uso de comentarios en el código fuente es relativamente sencillo y requiere de alguna metodología para mantener siempre la documentación del código actualizado. No es de extrañar encontrarse aplicaciones que no tienen ningún comentario y que, tras años de evolución, se convierte en una tarea muy complicada poder entender qué se está haciendo y por qué se ha realizado de esa forma. Por ello, **el uso de comentarios en el código es altamente recomendable.**

Hay que ser consciente de que cualquier modificación que se haga en el código también debe considerar la **actualización de los comentarios**. Una situación que suele ser muy común es encontrar un código con comentarios desactualizados, lo cual puede confundir enormemente a los desarrolladores y pensar que un código se comporta de una forma cuando realmente no es así.

Los **lenguajes de programación** permiten la introducción de comentarios fácilmente a través de comentarios de bloques o comentarios de línea.

Los comentarios son tan importantes que se suele decir que son parte también del código. Se puede deducir que comentar correctamente el código puede tener un 'coste' de mantenimiento a lo largo del tiempo.

Por ejemplo, hay que tener en cuenta qué sucede cuando aplicamos técnicas de refactorización.

En este caso, sería necesario actualizar los comentarios según corresponda a los cambios que hemos realizado.

No obstante, este tiempo empleado en **documentar el código**, pronto nos daremos cuenta que más que un coste, será una **inversión,** sobre todo, cuando solventemos rápidamente cualquier incongruencia en el código, y lo comparemos con todo el tiempo que se podría perder al intentar solucionar problemas de código indocumentados.



Fig. 3. La documentación debe generarse en el momento oportuno.

/ 7. Caso práctico 2: "Documentar siempre"

Planteamiento: Una vez que finalizamos el desarrollo del proyecto nos damos cuenta de que no tenemos incluido ningún comentario en el código fuente. En este momento nuestro jefe nos dice que no es importante, que hagamos la entrega que es lo importante, ya que el cliente lo espera para una fecha, y ya volveremos al código cuando se pueda.

Nudo: ¿Qué opinas de esta forma de trabajo?

Desenlace: Si observamos en detenimiento el enunciado planteado, vemos que pueden existir dos posibles errores. Por un lado, es necesario que cada vez que realicemos un cambio en el código éste quede automáticamente documentado para evitar llegar a fases muy avanzadas del proyecto sin tener mucha documentación.

Al trabajar con diferentes equipos puede ser posible que alguien quiera realizar un cambio en el código que tenemos en el repositorio y los cambios no estén documentados correctamente. Por otro lado, cuando se nos dice que realicemos la entrega sin documentar el código, puede ser un error grave ya que este código puede servir como base de documentación a los usuarios de nuestro producto.

Además, cuando en las empresas se comenta que "volveremos más adelante sobre esto", no siempre suele ser así y, en caso de volver, suele hacerse pasado bastante tiempo, cuando ya nos hemos olvidado de lo que hicimos sobre el proyecto, por lo que el tiempo requerido para poder crear la documentación será mucho más alto que si lo hacemos en el momento.

El motivo de esto es que, pasado un tiempo, normalmente de lo que nos olvidamos, sobre todo, es de la lógica que hemos seguido para desarrollar un problema determinado. Es más, si estamos asignados a múltiples desarrollos y proyectos, veremos que rápidamente olvidamos algunos detalles de algunos de los proyectos en los que estamos asignados.

/ 8. Resumen y resolución del caso práctico de la unidad

En este tema hemos presentado qué son los **sistemas de control de versiones** y el flujo de trabajo que debemos seguir con ellos para poder sincronizar diferentes equipos y diferentes desarrolladores en un mismo proyecto. Es importante saber qué hacer cuando muchas personas trabajan en el mismo proyecto, ya que fácilmente pueden ocurrir situaciones en las que se modifique el mismo archivo o bloque de código, y que tendrá que ser solucionado en el futuro.

Además, también hemos visto las **herramientas** para generar documentación y mantener el código bien comentado ya que se trata del primer nivel de documentación que generamos, es decir, documentamos para nosotros mismos, los desarrolladores. Por último, hemos presentado herramientas de generación de documentación directamente relacionada con la documentación del código y no con la generación de automática de manuales.

Resolución del caso práctico inicial

Según se nos indica en el caso práctico, nuestro cliente recibe continuamente quejas de los usuarios porque hay errores en el código tras nuestra actualización. Nuestra postura es que hemos utilizado un sistema de control de versiones y eso no puede generar ningún tipo de error.

Nuestra justificación no tiene sentido porque el hecho de utilizar un sistema de control de versiones no tiene nada que ver con los errores que se puedan producir. Es más, si no se utiliza correctamente el sistema de control de versiones podemos estar introduciendo errores a la hora de resolver los conflictos que puedan existir cuando subamos el código.

Para poder evitar ese tipo de errores tenemos que ser capaces de disponer de una batería de pruebas suficientemente potente y una documentación que nos ayude a poder detectar los errores fácilmente.



Fig. 4. Se necesita un uso correcto del SCV para que sea efectivo.

/ 9. Bibliografía

Price, J. & Korman, H. (1993). *How to communicate technical information: a handbook of software and hardware documentation*. Redwood City, Calif: Benjamin/Cummings Pub. Co.

Clements, P. (2011). *Documenting software architectures: views and beyond.* Upper Saddle River, NJ: Addison-Wesley.

Swicegood, T. & Pfalzer, S. (2012). *Pragmatic version control using Git*. Place of publication not identified: Pragmatic Bookshelf.